

HIDDEN OBJECTS GAME

basic manual

Version 1.0

Overview

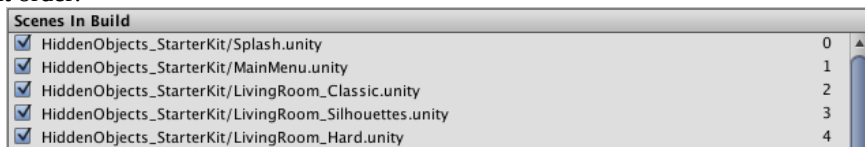
It is a complete solution to create a nice and professional Hidden Objects game really fast and easy. The package has everything you may need – active objects creating, menus visualizing, different types of hints, dialogs, etc. Every element was carefully designed to be effective, universal and easy to use. Game is fully optimized and Mobile-ready.

The Starter Kit provides you with a prepared game that can be easily customized and big amount of useful components/features to create your own gameplay:

- Gameplay logic and all related components
- Nice 2D art (LivingRoom scene, Blue Cartoon GUI, etc)
- Scripts for importing scenes and animation
- Easy Menu system
- Sound manager components
- Easy and nice dialogs sub-system
- etc

How to setup Demo-game

Basic preparations are really simple – just add all scenes to Scenes In Build menu (Unity Menu → File → Build Settings) in the next order:



Thats all – you can compile game for any platform or just try it in the Editor.

Basic components and their adjustments

Project has next list of basic objects/components needed for the game (check *_Scripts* folder):

- **ActiveObject** – Basic component for “hidden object”. Processes it reaction on click etc
- **HintObject + HintObject_Visualizer** – Process/visualize holders of “hidden objects” list
- **ObjectsInitializer** – OnStart initializes all Objects and generates lists of “hidden objects”
- **GameManager** – Responsible for processing game rules/tasks
- **HUDManager** – Handles basic User interface
- **SoundManager** – Controls all sounds
- **Hint + Hint_FlyingIndicator** – Allows to help player by hinting on “hidden object”
- **SimpleDialog** - Processes simple dialogue functionality
- **LevelsUnlocker** - Handles levels unlocking if stored "TotalScore" is bigger than level cost

Most of basic game objects has prefabs (please check *_Prefabs* folder) – this should help you a lot, since you can just drag'n'drop them to your scene.

You may want adjust their behavior or appearance – for this you should either update related values or change Model/Material/etc in the scene.

If you want to get/change any of game asset – you can find their sources in the *_Assets* folder.

*If you have any questions – please don't hesitate to contact me (e-mail: AllebiGames@gmail.com)
I'm always happy to help you!*

How to create/import new scene

First of all – you can simple create new 2D scene using standard Unity tools and objects. There are only 2 strict requirements: all “active objects” should have common parent and each of them must have attached Collider.

If you want to import new scene - the easiest way is to create image(.PSD as instance) with layers (each piece should be in separate layer) end then export it to bunch of images + data-file.

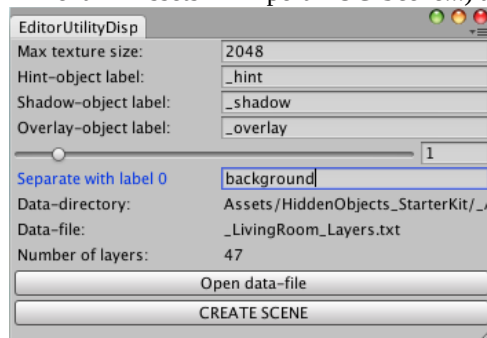
It's recommended to have postfixes(like “_shadow”) in layers names, those allows to specify is layer overlay, shadow, ActiveObject or HintObject.

I've prepared a small GIMP script for you (please check `_GIMP_ExtractAllLayers.chm` in Scripts/Utility folder) - to export such image from free "The Gimp" (<http://www.gimp.org/>) editor really fast and easy.

The script will create folder with each layer exported as .png and data file to parse. Drag this folder to your Project window in Unity.

The Gimp can open PSD files, so if needed - you can save your image to .PSD from Photoshop and then open in GIMP to use exporting script.

Kit has built-in script (`_Scripts/_Editor/_SceneImporter.js`) to create atlas and generate scene using this data. You can run it from Unity menu (Main Menu -> Assets -> Import HOG Scene...) and select text data file:



In the utility window you also can specify:

- Max size of generated atlas
- Postfix for special objects (hint, shadow, overlay)
- Quantity and names of layers/images, that should be excluded from atlas and created with separated material

Info for manual exporting (just in case) :

- each layer/piece should be saved to separated .png file
- info about all layers must be saved to text file with structure:
 - `Layer_3.png` - layer/file name
 - `723` - X position of layer in the image
 - `790` - Y position of layer in the image
 - `75` - X size of layer (width)
 - `91` - Y size of layer (height)
 - `-----` - separator-string before information about next layer

How to setup scene functionality

Using Prefabs and scene-exporter (described above) you can setup everything just in few simple steps:

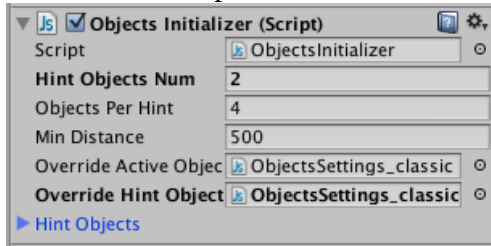
- Add **ObjectInitializer** component/script to the parent object of all functional objects
- Assign to `OverrideActiveObjectSettings` and `OverrideHintObjectVizualizer` properties of **ObjectInitializer** “ObjectsSettings_classic” prefab from `_Prefabs` folder
- Drag and drop to scene “MainCamera” prefab from `_Prefabs` folder
- Adjust attached components(GameManager, Hint, etc) as you want

Fully manual setup is more complicated:

- Attach *ActiveObject* component to each object you want to be “hidden”
- Attach *HintObject* and *HinObject_Vizualizer* components to each object you want to hold list of “hidden objects”
- Add **ObjectInitializer** component/script to the parent object of all functional objects
- Assign to `OverrideActiveObjectSettings` and `OverrideHintObjectVizualizer` properties of template object with attached and adjusted *ActiveObject* and *HinObject_Vizualizer* components
- Add to *MainCamera* object (and adjust) at least next components GameManager, HUDManager, Sound Manager

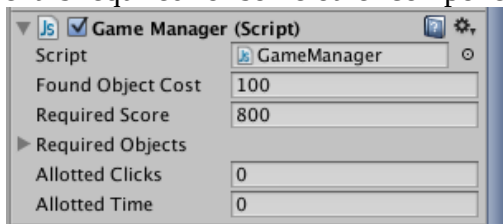
Main components

ObjectsInitializer - Prepare and initialize all objects (objects holds list of Hidden objects)



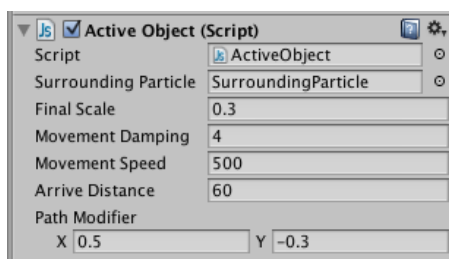
- | | |
|---------------------------------------|---|
| hintObjectsNum | - How many active HintObjects can be in scene |
| objectsPerHint | - How many Active(hidden) objects should be assigned to each HintObject |
| minDistance | - Minimal distance between Hint object and ActiveObject assigned to it |
| overrideActiveObjectsSettings | - Override ActiveObjects settings by this reference |
| overrideHintObjectsVisualizers | - Override HintObjects Visualizers by this reference |
| hintObjects | - List of chosen hint objects |

GameManager - Script contains and processes Win/Lose conditions for the game. This component is required for some other components - please checks cripts and/or warning messages



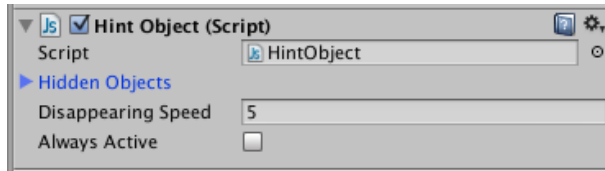
- | | |
|------------------------|--|
| foundObjectCost | - How much one found object costs |
| requiredScore | - How many score should player has to Win |
| requiredObjects | - List of mandatory objects, those player should find to Win |
| allottedClicks | - How many clicks player has to execute Win-conditions (before GameOver) |
| allottedTime | - How much time player has to execute Win-conditions (before GameOver) |

ActiveObject - Script controls active objects (those can be find)



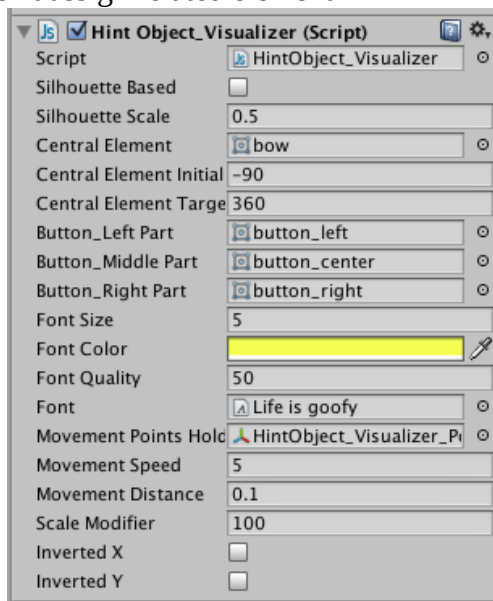
- | | |
|----------------------------|--|
| SurroundingParticle | - Prefab for particle surrounding found objects movement |
| finalScale | - Final scale of the object |
| movementDamping | - Smooth facing/movement value |
| movementSpeed | - Speed of object movement along the path |
| arriveDistance | - How far should object be to waypoint for its activation and choosing new |
| pathModifier | - Path parabolas modifier |

HintObject - Contains list of ActiveObjects to find and allows to visualize it



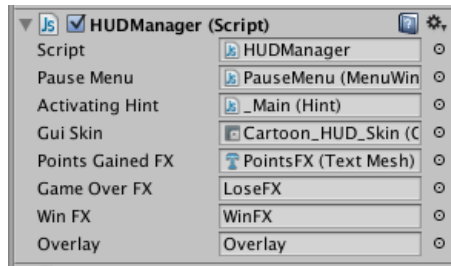
- hiddenObjects** - List of ActiveObjects (objects to find) assigned to this Hint holder
- disappearingSpeed** - Disappearing speed - when all objects have been found
- alwaysActive** - Hint will be activated and hiddenObjects-list will be shown OnStart

HintObject Visualizer - Script allows to create and process animated visualization of HintObject hiddenObjects list. All elements are optional, so if you don't want to have some of parts- just don't assign related element



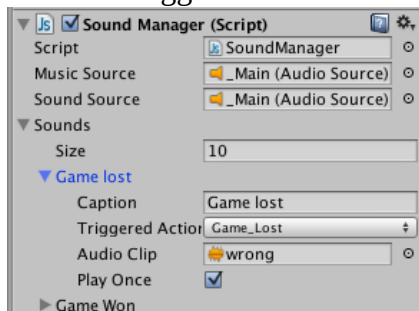
- silhouetteBased** - Should hiddenObjects be visualized as silhouettes or by **description**
- silhouetteScale** - Custom scale for silhouettes
- centralElement** - Animated Central element of visualization
- centralElementInitialRotation** - Initial rotation of the Central element. At appearing Rotation will be animated from this to TargetRotation
- centralElementTargetRotation** - Target rotation of the Central element. At appearing Rotation will be animated from InitialRotation to this
- button_LeftPart** - Left part of visual ListElement background
- button_MiddlePart** - Middle part of visual ListElement background
- button_RightPart** - Right part of visual ListElement background
- fontSize** - Caption(description) font size
- fontColor** - Caption(description) color
- fontQuality** - Caption(description) font quality
- font** - Caption(description) font
- movementPointsHolder** - Link to reference object that children positions will be used as template for listElements positioning
- movementSpeed** - How fast listElements will be positioning
- movementDistance** - How far from target position can listElements be
- scaleModifier** - Scale modifier for whole visualization
- invertedX** - Should be X-position of listElements inverted related to positions in movementPointsHolder
- invertedY** - Should be Y-position of listElements inverted related to positions in movementPointsHolder

HUDManager - Script processes and draw whole common GUI



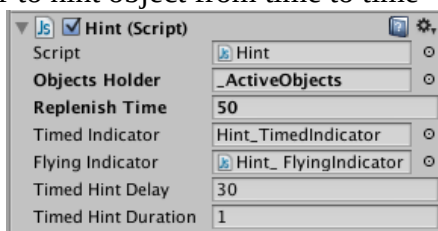
- pauseMenu** - Link to PauseMenu MenuWindow
- activatingHint** - Link to activatingHint object
- guiSkin** - GuiSkin to use
- pointsGainedFX** - Link to D text with text and effect, those will be shown when one more object found
- GameOverFX** - Link to Object with Win effect
- WinFX** - Link to Object with GameOver effect
- overlay** - Link to Overlay object

SoundManager - Script allows to assign and automatically process all event-related(triggerred) sound. Based on triggeredAction value of gameManager



- musicSource** - Link to AudioSource handles music
- soundSource** - Link to AudioSource handles sound effects
- sounds** - List of GameSounds to process

Hint - Script manage hints Flying - that can be activated to find and point object. Timed - that appear to hint object from time to time



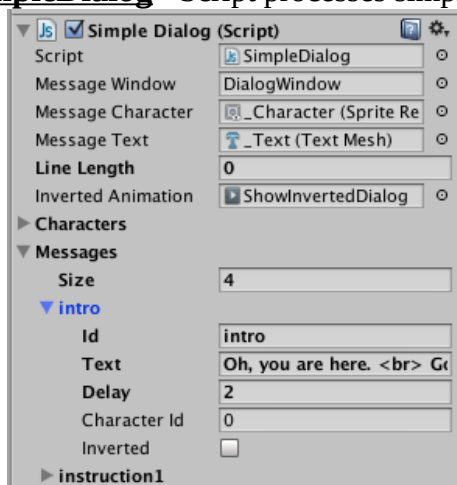
- objectsHolder** - Object that holds all active/hint objects as children
- replenishTime** - How often flying hint can be used
- timedIndicator** - Object to visualize timed hint
- flyingIndicator** - Object to visualize flying hint
- timedHintDelay** - Delay till timed hint activation
- timedHintDuration** - Duration of timed hint visibility

Hint FlyingIndicator - Script manage activating hint indicator movement and effects, that can find and point object



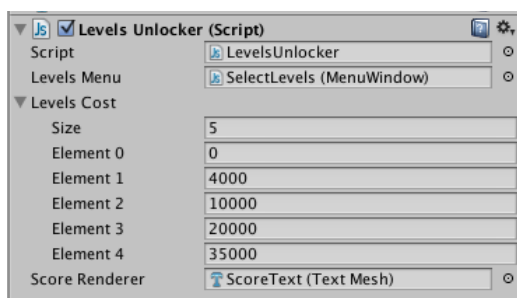
- appearFX** - Effect to be enabled on enable
- idleFX** - Effect to be enabled in idle movement
- hideFX** - Effect to be enabled at hiding
- movementTime** - Time of object movement along the path
- movementRandomizer** - Parameter to randomize movement

SimpleDialog - Script processes simple dialogs functionality



- messageWindow** - Link to Dialog visualization object
- messageCharacter** - Link to sprite dedicated for character rendering
- messageText** - Link to message text mesh
- lineLength** - Number of symbols in line (before World-wrap)
- invertedAnimation** - Link to animation to show/make inverted visualization object position
- characters** - List of characters sprites
- messages** - List of all messages

LevelsUnlocker - This script unlocking levels if stored "TotalScore" is bigger than level cost



- levelsMenu** - Link to levels window
- levelsCost** - List of unlocking cost for each level
- scoreRenderer** - Link to D text to render score